M11: Services and Systemd: Keep It Running

> "Nginx is installed. But it's not running. And if the server reboots, it won't start. Make it run — now and forever."

Quick Check

Show of hands:

- Who has installed software that didn't automatically start?
- Who has wondered what "systemd" actually is?
- Who has had a service crash and not restart?

> Today you'll learn the system that manages every background process on modern Linux — the same service management concepts used by Docker, Kubernetes, and cloud orchestration platforms.

What You'll Walk Away With

By the end of this mission, you'll be able to:

1. **Understand Services**
   Understand what services are and why they run in background

2. **Control Services**
   Start, stop, and restart services with systemctl

3. **Boot Behavior**
   Configure boot behavior (enable/disable services)

4. **Check Status**
   Check service status and read service output

5. **Read Logs**
   Read service logs with journalctl

6. **Unit Files**
   Understand unit files (the service configuration)

Commands you'll learn: `systemctl status`, `systemctl start/stop/restart`, `systemctl enable/disable`, `journalctl`

Software Installed ≠ Software Running

Installing a package puts files on disk. But a **web server** or **database** needs to run continuously.

A service (daemon) is a program that:

- Runs in the background
- Starts automatically on boot
- Restarts if it crashes
- Listens on a network port

1

Package installed

```
sudo apt install nginx
```

2

Service started

```
sudo systemctl start nginx
```

3

Application running

```
curl http://localhost → Welcome to nginx!
```

What Is systemd?

systemd is the system and service manager for modern Linux.

Brief history:

Era	Init System	Notes
Traditional	SysV init	Shell scripts
~2006	Upstart	Event-based
~2015+	**systemd**	Universal, modern

What systemd manages:

- Services (nginx, sshd, postgresql)
- Timers (like cron but better)
- Logging (journald)
- Network interfaces

The main command: `systemctl`

systemctl status — Is It Running?

What it does: Shows current state, recent logs, and process info.

Syntax: `systemctl status servicename`

```
systemctl status nginx
● nginx.service - A high performance web server
     Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
     Active: active (running) since Thu 2026-01-30 10:00:00 UTC
   Main PID: 1234 (nginx)
      Tasks: 2
     Memory: 5.0M
```

Reading the status:

Field	Meaning
`loaded`	Unit file found
`enabled`	Starts on boot
`active (running)`	Currently running
`Main PID: 1234`	Process ID

systemctl start / stop / restart

What they do: Control the runtime state of a service.

```
# Start a service
sudo systemctl start nginx

# Stop a service
sudo systemctl stop nginx

# Restart (stop + start)
sudo systemctl restart nginx

# Reload config (no downtime!)
sudo systemctl reload nginx
```

reload vs restart:

Action	What Happens	Downtime?
`restart`	Stop → Start	Brief
`reload`	Re-read config	**None**

systemctl enable / disable — Boot Behavior

What they do: Control whether service starts on boot.

```
# Enable service to start on boot
sudo systemctl enable nginx

# Disable from starting on boot
sudo systemctl disable nginx

# Enable AND start immediately
sudo systemctl enable --now nginx

# Disable AND stop immediately
sudo systemctl disable --now nginx
```

Quick checks:

```
# Is it running RIGHT NOW?
systemctl is-active nginx
active

# Will it start on BOOT?
systemctl is-enabled nginx
enabled
```

systemctl list-units — See All Services

What it does: Lists all active service units.

```
# List all active services
systemctl list-units --type=service
UNIT                    LOAD   ACTIVE SUB     DESCRIPTION
cron.service            loaded active running Background program
nginx.service           loaded active running Web server
ssh.service             loaded active running SSH server

# List all services (including inactive)
systemctl list-units --type=service --all

# List only failed services
systemctl list-units --type=service --state=failed
```

journalctl — Reading Service Logs

What it does: Queries the systemd journal (centralized logging).

```
# All logs for specific service
sudo journalctl -u nginx

# Follow logs in real-time (like tail -f)
sudo journalctl -u nginx -f

# Last 50 lines
sudo journalctl -u nginx -n 50

# Logs since last boot
sudo journalctl -u nginx -b

# Logs from specific time
sudo journalctl -u nginx --since "2026-01-30 10:00"
```

> **This is how you debug "why won't my service start?"** — always check `journalctl -u servicename`

journalctl Key Options

Option	Purpose
`-u service`	Filter by service name
`-f`	Follow (real-time stream)
`-n N`	Last N lines
`-b`	Since last boot
`--since "datetime"`	From specific time
`-p err`	Show errors only

Priority levels:

`emerg`, `alert`, `crit`, `err`, `warning`, `notice`, `info`, `de`

```
# Show only errors
sudo journalctl -u nginx -p err
```

Anatomy of a systemd Unit File

Where they live: `/lib/systemd/system/` (system) or `/etc/systemd/system/` (custom)

```
cat /lib/systemd/system/nginx.service
```

```
[Unit]
Description=A high performance web server
After=network.target

[Service]
Type=forking
ExecStart=/usr/sbin/nginx
ExecReload=/usr/sbin/nginx -s reload
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Understanding Unit File Sections

Section	Purpose	Key Directives
`[Unit]`	Description, dependencies	`After`, `Requires`, `Wants`
`[Service]`	How to run the service	`ExecStart`, `Restart`, `User`
`[Install]`	When to start	`WantedBy=multi-user.target`

Common Service directives:

Directive	Purpose
`ExecStart`	Command to start
`ExecStop`	Command to stop
`Restart=on-failure`	Auto-restart if crashes
`User=www-data`	Run as specific user

In the Cloud: IAM vs OS Users

You Need Both!

IAM Users (AWS/GCP)	OS Users (Linux)
Access cloud console	Access server via SSH
Create resources	Run commands
Manage cloud services	Install software
Billing access	Configure apps

How they connect:

```
[AWS Console]                [EC2 Instance]
IAM User: anas@company       OS User: anas
Permission: ec2:*            Permission: sudo
     |                            |
     +--- Creates instance -----> +--- Configures nginx
     +--- Manages security        +--- Manages users
```

Aha! Moment 💡

Think about what a service manager does:

01

Starts processes

Starts processes and keeps them running

02

Restarts on failure

Restarts on failure automatically

03

Manages dependencies

Manages dependencies (database before app)

04

Collects logs

Collects logs in one place

05

Enables on boot

Enables on boot for high availability

Now look at container orchestration:

systemd Concept	Docker/K8s Equivalent	Same Problem
`systemctl start`	`docker run` / `kubectl apply`	Start a process
`Restart=on-failure`	Docker restart policies / K8s liveness probes	Auto-restart crashes
`After=network.target`	`depends_on` / K8s init containers	Dependency order
`journalctl -u service`	`docker logs` / `kubectl logs`	View output
`systemctl enable`	K8s Deployment replicas / ECS desired count	Run forever

Real-world evolution:

```
Traditional:
- systemd manages nginx on one server
- Restart if crashed: Restart=on-failure

Cloud-native:
- Kubernetes manages nginx on 100 servers
- Restart if crashed: livenessProbe
- Same concept, bigger scale
```

> systemd solved "how do we keep services running?" in 2010. Every orchestration tool since has copied the homework.

Systemd & Service Management Cheat Sheet

Checking service status:

```
# Detailed status
systemctl status nginx

# Is it running RIGHT NOW?
systemctl is-active nginx
active

# Will it start on BOOT?
systemctl is-enabled nginx
enabled

# Is it failed?
systemctl is-failed nginx
```

Controlling services:

```
# Start service
sudo systemctl start nginx

# Stop service
sudo systemctl stop nginx

# Restart (stop + start)
sudo systemctl restart nginx

# Reload config (no downtime)
sudo systemctl reload nginx

# Restart if running, start if stopped
sudo systemctl try-restart nginx
```

Boot behavior:

```
# Enable (start on boot)
sudo systemctl enable nginx

# Disable (don't start on boot)
sudo systemctl disable nginx

# Enable AND start immediately
sudo systemctl enable --now nginx

# Disable AND stop immediately
sudo systemctl disable --now nginx
```

Listing services:

```
# All active services
systemctl list-units --type=service

# All services (including inactive)
systemctl list-units --type=service --all

# Only failed services
systemctl list-units --type=service --state=failed

# Only enabled services
systemctl list-unit-files --type=service --state=enabled
```

Reading logs with journalctl:

```
# All logs for service
sudo journalctl -u nginx

# Follow logs in real-time
sudo journalctl -u nginx -f

# Last N lines
sudo journalctl -u nginx -n 50

# Since last boot
sudo journalctl -u nginx -b

# Since specific time
sudo journalctl -u nginx --since "2026-02-10 10:00"
sudo journalctl -u nginx --since "1 hour ago"
sudo journalctl -u nginx --since yesterday

# Until specific time
sudo journalctl -u nginx --until "2026-02-10 11:00"

# Time range
sudo journalctl -u nginx --since "10:00" --until "11:00"

# Only errors
sudo journalctl -u nginx -p err

# Multiple services
sudo journalctl -u nginx -u postgresql

# No pager (all output)
sudo journalctl -u nginx --no-pager
```

Mission Complete: Services and Systemd ✅

You mastered service management:

Services

Understood what services are and why they matter

Control

Started and stopped services with systemctl

Boot

Configured boot behavior with enable/disable

Status

Checked service status and health

Logs

Read service logs with journalctl

Unit Files

Understood unit file structure

Commands you now know: `systemctl status/start/stop/restart/enable/disable`, `journalctl`

> **Day 2 Complete!** You've built a production-ready server from scratch. You created users, granted precise permissions with sudo, locked down SSH, installed software with APT, and configured services to run forever. Tomorrow: Automation. Because everything you just did manually? You're about to script it.

Questions?